

Day06 Part C 学习文档 v1.0 : Memory Lifecycle (记忆生命周期)

本文是《从零实现 Agent Runtime》学习阶段的 Day06 Part C 正式学习文档。

Part A 建立了 Memory 的基础模型：Memory 不是聊天记录，而是 Runtime 从历史交互中提取出来的长期 State。Part B 进一步建立了 Memory Architecture：Memory System 不是 Vector DB，而是由写入链、读取链、Retriever、Ranker、Context Builder 共同组成的长期信息管理系统。

Part C 继续回答：

一条 Memory 从被创建，到被更新、合并、衰减、遗忘，中间到底如何被 Runtime 管理？

本节定位

Day06 Part C 关注 Memory System 的生命周期层。

如果 Part B 解决的是：

Memory System 怎么存、怎么找、怎么排序、怎么进入 Context？

那么 Part C 解决的是：

Memory 是怎么出生、变化、冲突、衰减和退出当前有效状态的？

本节核心结论是：

Memory Lifecycle 不是简单的 Create / Update / Delete，而是 Runtime 持续对长期 State 做 Reconciliation。Memory 是带有 Identity、Scope、Lifecycle 和 History 的 Long-term State；Create / Update / Merge / Decay / Forget 都是状态协调与生命周期管理的一部分。

目录

为什么 Memory 需要 Lifecycle

Memory 是 State，不是 Event

Memory Create Decision

Conversation、Candidate 与 Memory 的区别

Rule、LLM 与 Runtime 的三层 Create 决策

Importance、Confidence 与 Scope

Memory Update / Merge Decision

Similarity 与 State Identity

State Slot、Entity、Scope 与 Temporal Signal

Update、Merge、Conflict 的边界

Memory Decay / Forget

Confidence、Importance、Recency 的分工

Forget 不等于 Physical Delete

Lifecycle Engine 的工业级实现

Current State + History

Memory Lifecycle、Retrieval 与 Projection 的边界

Memory 是否可以看成 Agent

Part C 最终模型

本 Part 核心知识点

写书 TODO

写书素材

本 Part 核心认知升级

工业级实现

知识地图

面试视角

本章思考题

前置问题回收

下一节学习计划

为什么 Memory 需要 Lifecycle

如果 Memory 只是：

Conversation

↓

抽取

↓

永久保存

那么系统最终一定会出现状态冲突。

例如用户今天说：

我最近喜欢 Vue。

系统保存：

User prefers Vue

一个月后用户说：

我现在主要写 React 了。

系统又保存：

User prefers React

如果 Memory Store 只是 append：

Memory Store

1. User prefers Vue
2. User prefers React

下一次用户问：

我应该用什么前端技术？

Retriever 可能同时召回：

Vue
React

问题就变成：

到底哪个才是当前有效状态？

所以 Memory 最大的问题不是“能不能存下来”，而是：

过去的事实如何随着时间和新信息变化？

这就是 Lifecycle。

Memory 是 State，不是 Event

Conversation 更像 Event Log：

User said Vue
User said React
User said TypeScript

Memory 更应该表示当前长期 State：

Current preference:
frontend_framework = React

所以：

Conversation
↓
Event

Memory
↓
Current State

Memory 必须有 Lifecycle，是因为：

State 会变化。

一个基础生命周期可以先理解成：

Candidate
|
v
Create
|
v
Active
|
+-----> Update
|
+-----> Merge
|
+-----> Decay
|
v
Deprecated / Forgotten

这也是 Part C 和 Part B 的分工：

Part B = Information Retrieval
Part C = State Lifecycle

两者合起来，Memory System 才完整。

Memory Create Decision

Create 的核心问题不是：

用户说了一句话
↓
创建 Memory

而是：

Conversation / Observation
|
v
Memory Candidate
|
v
Should Remember?

例如：

今天天气不错。

通常不值得保存为长期 Memory。

而：

我以后希望你回答 Agent 问题时，多解释底层原理。

很可能产生：

type = preference
content = User prefers principle-oriented explanations

再比如：

我现在正在做一个 TypeScript 的 Agent Runtime 项目。

可能产生：

type = project_context
content = User is building a TypeScript Agent Runtime

所以 Create 实际上是：

判断一个 Observation 是否具有长期 State 价值。

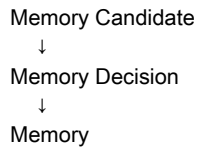
可以抽象成：

Observation
↓
Extraction
↓
Candidate Memory
↓
Importance / Confidence / Scope
↓
Should Create?

Conversation、Candidate 与 Memory 的区别

这里要把三个层次分清楚：

Conversation
↓
Observation
↓



例如用户说：

我现在主要写 TypeScript，而且以后讲 Agent 的时候希望你多解释底层原理。

整个 Observation 是一句话，但它可能被抽成两个 Candidate：

```
Candidate A
type = technical_profile
content = User mainly uses TypeScript

Candidate B
type = preference
content = User prefers principle-oriented explanations
```

然后再分别判断：

```
Candidate A → Create
Candidate B → Create
```

所以：

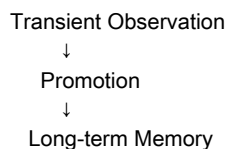
Memory Extraction 不等于 Memory Create。

Extraction 只说明系统从 Conversation 中抽出了可能值得记住的信息；Create 说明 Runtime 认为它具备长期 State 资格。

Create 的真正意义是：

把某些原本属于当前 Context 的信息，提升为跨 Task / 跨 Session 可以复用的 State。

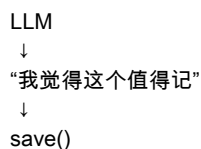
也就是：



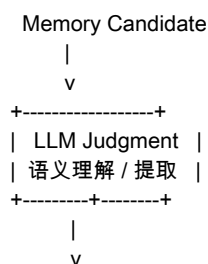
因此 Memory Create 本质上是一次 State Promotion。

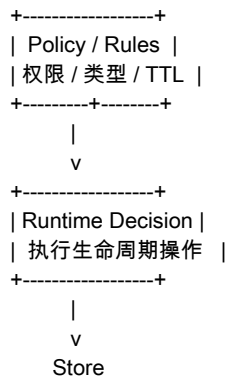
Rule、LLM 与 Runtime 的三层 Create 决策

Create 不应该简单变成：



更成熟的系统通常是：





三者分工是：

LLM
→ “它可能是什么？”

Policy
→ “它允许不允许？”

Runtime
→ “那我具体做什么？”

纯 Rule 的优点是稳定、便宜、可预测、容易测试；问题是自然语言表达非常多样，规则很难覆盖。

纯 LLM 的优点是语义理解灵活；问题是成本高、不稳定、不可控，尤其不能把敏感数据、权限信息、临时业务状态的保存决定完全交给模型。

所以工业级 Memory 更合理的是：

LLM 负责理解，Policy 负责约束，Runtime 负责执行。

这个思想和 Day05 Tool Calling / Permission / Human Approval 一致：

Tool Call
≠
Tool Execution

Memory Decision
≠
Memory Mutation

模型提出意图，Runtime 执行状态变化。

Importance、Confidence 与 Scope

Create Decision 至少要判断：

Long-term?
Reusable?
Scoped?
Sensitive?
Existing?

其中 importance、confidence、scope 是非常关键的字段。

Importance

Importance 回答：

这条 Memory 值不值得长期保留？

例如：

User prefers principle-oriented explanations

长期价值较高。

而：

User is currently debugging line 327

对当前 Task 可能重要，但长期价值可能很低。

Confidence

Confidence 回答：

我有多确定这条 Memory 是真的？

例如：

“我应该比较喜欢 React 吧。”

可能是：

importance = 0.7
confidence = 0.5

而：

“以后请全部用 React 示例，我不想再看 Vue 代码。”

可能是：

importance = high
confidence = high

Importance 和 Confidence 不能混成一个 score：

Importance
= 值不值得记？

Confidence
= 我有多确定这是真的？

Scope

Scope 回答：

这条 Memory 对谁、对什么范围成立？

例如用户说：

这个项目我用 Vue。

不能简单生成：

User prefers Vue.

更合理的是：

type = technical_profile
scope = project_123
content = Project uses Vue

否则下一次用户问新项目技术选型时，Agent 可能把 Project A 的技术栈错误理解为用户全局偏好。

这就是 Memory 污染。

Create Decision 不只是：

Should Remember?

还包括：

Remember For Whom?
Remember For What Scope?

Memory Update / Merge Decision

有了 Memory 之后，新信息进来时，Runtime 需要判断：

- Create
- Update
- Merge
- Conflict
- Ignore

最经典的例子：

Old:
User prefers Vue.

New:
User prefers React.

如果只看文本或向量，相似度很可能很高：

- Vue
- React
- frontend
- framework

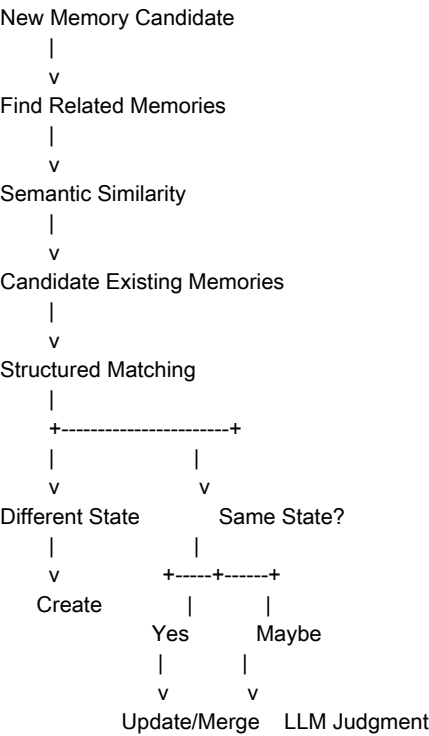
但这只能说明：

它们讨论的是相近主题。

不能证明：

它们是同一个 Memory Slot。

真正的判断链路应该是：



Retriever 找到“可能相关”的旧 Memory，不等于 Runtime 已经决定“它们是同一条 State”。

Similarity 与 State Identity

Similarity 只能回答：

两段内容语义上像不像、相关不相关？

它不能直接回答：

它们是不是同一个长期 State？

例如：

Memory A:
User works with TypeScript.

Memory B:
User is building a TypeScript Agent Runtime.

Memory C:
Project X uses TypeScript.

三者语义相似，但它们不是同一个 State：

A = User Profile
B = Project / Skill Context
C = Project-scoped Technology

所以：

Semantic Relatedness
≠
State Identity

这就是 Memory Lifecycle 的核心边界。

Vector Search 负责：

“它们很像”

Memory Lifecycle 负责：

“它们到底应该发生什么关系”

State Slot、Entity、Scope 与 Temporal Signal

判断是否 Update，不能只存：

content = User prefers Vue

更应该尽量保留：

type = preference
slot = frontend_framework
scope = user
content = User prefers Vue

当新信息出现：

type = preference
slot = frontend_framework
scope = user
content = User prefers React

Runtime 就可以判断：

type = same
slot = same

scope = same

这很可能是同一个 State Slot 的新值。

Slot 可以理解成：

一个 Memory 所描述的状态位置。

例如：

```
frontend_framework
primary_language
favorite_editor
current_project
communication_style
response_preference
```

于是：

```
frontend_framework = Vue
```

后来：

```
frontend_framework = React
```

就是：

同一个 Slot
不同的 Value

Entity 则回答：

这个状态属于谁或什么对象？

例如：

```
Entity = user
Slot = frontend_framework
Value = React
```

或者：

```
Entity = project_A
Slot = frontend_framework
Value = Vue
```

真正判断“是不是同一 Memory”，是在判断：

(Entity, Type, Slot, Scope)

是否一致或高度一致。

Temporal Signal 也很关键。

2024:
User prefers Vue.

2026:
User prefers React.

时间不是简单的“越新越重要”，而是帮助 Runtime 判断：

新事实是否对旧状态形成了时间上的替代关系。

Update、Merge、Conflict 的边界

Update

Update 可以定义为：

同一个 State Slot 的当前值发生改变。

例如：

Old:
type = preference
slot = explanation_style
value = concise

New:
type = preference
slot = explanation_style
value = detailed

结果是：

explanation_style = detailed

旧值可以被标记为 deprecated，或者进入历史记录。

Merge

Merge 强调：

多条 Memory 合起来后，形成一个更完整、更准确的长期状态。

例如：

Memory A:
User works remotely.

Memory B:
User usually works remotely from Japan.

更合理的结果可能是：

User works remotely from Japan.

所以：

Update
= replace / revise

Merge
= combine / enrich

Conflict

除了 Create / Update / Merge，还有一种情况是无法确定。

例如：

Old:
User prefers React

New:
I also really like Vue.

这不能直接变成：

React → Vue

因为喜欢 Vue 不一定意味着不喜欢 React。它可能是：

React + Vue

也可能需要更细的 Slot 或 Scope。

所以 Conflict 应该理解成：

当前信息不足以确定 State Relationship。

而不是：

数据库坏了。

Memory Lifecycle 本质上是 State Reconciliation。

Memory Decay / Forget

Memory 虽然是 Long-term State，但：

Long-term
≠
Forever Correct

例如：

2024:
User prefers Vue

2026:
User mainly uses React

旧 Memory 并没有因为“曾经是真的”就永远正确。

Decay 是：

Memory
↓
仍然存在
↓
但影响力逐渐下降

也就是：

Active
↓
Less Relevant
↓
Weak
↓
Deprecated
↓
Forgotten

Decay 不是 Delete。

更准确地说：

Decay 是 Memory 有效性下降的过程，Forget 是生命周期上的最终决策。

Confidence、Importance、Recency 的分工

这三个概念不能混。

Confidence
→ 可信不可信

Importance
→ 值不值得长期保留

Recency

→ 新不新鲜、最近有没有被创建、更新或使用

可以用概念模型理解：

Memory Utility
≈
Confidence
×
Importance
×
Recency

但这只是帮助理解，不代表工业系统一定使用这个精确公式。

一条 Memory 可能是：

Confidence = high
Importance = high
Recency = low

例如：

用户以前长期使用 Vue。

它可能可信、重要，但已经很旧。

所以它不是错误 Memory，而是：

一条历史有效、当前可能已经不够新鲜的 Memory。

不能简单“越旧越删除”。

例如：

User birthday = 1990-01-01

十年后 Recency 很低，但这不代表它应该被遗忘。

不同 Memory Type 的生命周期不同：

User profile
↓
通常比较稳定

User preference
↓
可能变化

Project context
↓
项目结束后价值下降

Task state
↓
任务结束后可能迅速失效

所以 Decay 不是一个全局统一的过期时间。

更合理的是：

Decay Signal
=
Type
+
Recency
+
Importance
+
Confidence

+
Access Pattern
+
Explicit Contradiction
+
Policy

Explicit Contradiction 往往比时间过期更重要。

例如刚保存 10 天的：

User prefers Vue

如果用户明确说：

I've completely switched to React.

那么最强信号不是 Recency，而是：

Explicit Contradiction

成熟的 Memory Lifecycle 应该由：

Time-based
+
Event-based
+
Semantic-based

共同驱动。

Forget 不等于 Physical Delete

很多人理解 Memory Forget：

Forget
↓
DELETE FROM memory

但工业系统通常需要区分：

active
deprecated
forgotten
deleted

例如：

User prefers Vue

后来：

User prefers React

更合理的操作可能是：

Vue
↓
deprecated

而不是物理删除。

原因包括：

- 审计：以前是什么、为什么改变、什么时候改变
- Debug：为什么 Agent 某次还认为用户喜欢 Vue

- Rebuild : 以后换了 Memory 策略，是否要重新计算
- Conflict Resolution : 为什么 Vue 和 React 曾经同时存在

所以：

Logical Forget
≠
Physical Delete

更准确地说：

Forget
= 不再把它当作当前有效 Memory 使用

Delete
= 物理移除数据

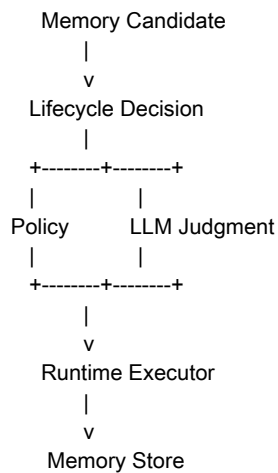
Lifecycle Engine 的工业级实现

工业实现首先要避免把所有生命周期逻辑塞进 Memory Entity。

不建议设计成：

```
Memory Entity
|
+-- create()
+-- update()
+-- merge()
+-- decay()
+-- forget()
```

更合理的是：



核心思想是：

Memory Store 保存 State，Lifecycle Engine 决定 State 怎么变化。

Lifecycle Decision 的输入是：

Existing Memories
+
New Observation
+
Current Runtime State
+
Policy

输出是：

Create
Update
Merge
Keep
Decay
Forget
Conflict

LLM、Rule、Runtime 的分工是：

LLM
→ Semantic Judgment

Policy / Rule
→ Hard Constraints

Runtime
→ State Transition

LLM 输出：

```
{  
  "action": "update",  
  "memory_id": "mem_123",  
  "new_value": "React"  
}
```

并不代表 Runtime 可以直接执行。

Runtime 还要检查：

memory_id 是否属于当前 user？
scope 是否正确？
permission 是否允许？
candidate confidence 是否满足阈值？
是否存在并发更新？
是否违反 lifecycle policy？

所以：

LLM Output
+
State Mutation

模型提出意图，Runtime 执行状态变化。

Current State + History

用户提出了一个非常关键的问题：

Memory 不是 Vector DB，那落库后的数据如果 Update，会留日志存之前的数据吗？

结论是：

可以，而且工业实现里通常非常有价值；但不是 Memory 必须如此。

Memory Update 背后可以分成两层：

Current State
+
History / Audit Log

例如最开始：

Memory:
frontend_framework = Vue

后来：

User now prefers React

当前有效状态变成：

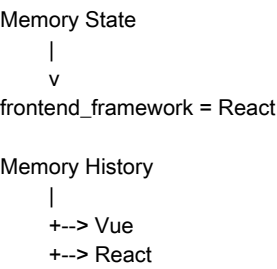
frontend_framework = React

但历史可以记录：

2026-08-01
Vue

2026-08-13
React

于是：



Current State 回答：

现在是什么？

History 回答：

为什么现在是这样？

实现 A：直接覆盖 + 审计日志

当前表：

memory

id
type
slot
content
status
updated_at

审计表：

memory_audit_log

memory_id
action
before
after
timestamp
reason

例如：

UPDATE

before:
Vue

after:
React

```
reason:
explicit_user_preference
```

这是简单、常见、实用的方案。

实现 B : Event Sourcing 风格

更进一步，不把 Update 当成真正的数据覆盖，而是保存：

```
MemoryCreated
MemoryUpdated
MemoryMerged
MemoryDeprecated
MemoryForgotten
```

然后：

```
Event Log
↓
Replay
↓
Current Memory State
```

但不是所有 Agent Memory 都需要完整 Event Sourcing。

简单 Agent 可以先用：

```
Current State + Audit Log
```

复杂、高审计要求的系统再考虑：

```
Event Log
↓
Projection / Reducer
↓
Current State
```

不要为了“工业级”第一版就直接上 Event Sourcing。

Memory Lifecycle、Retrieval 与 Projection 的边界

Part C 收尾时最值得补充的边界是：

```
Lifecycle
→ 这条 Memory 现在还成立吗？

Retrieval
→ 这条成立的 Memory 对当前 Query 有用吗？

Projection
→ 应该以什么形式告诉 LLM？
```

例如：

```
Memory:
User uses TypeScript
```

它可能：

```
Lifecycle = active
```

但当前任务是：

```
帮我分析 Java GC
```

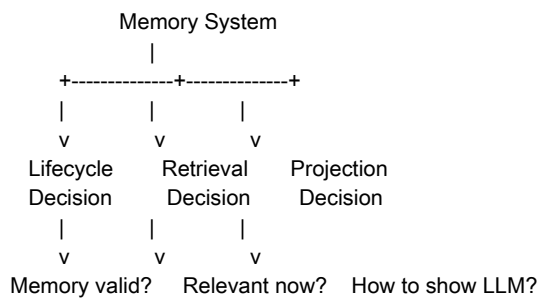
那么：

```
Retrieval relevance = low
```

所以：

Active
≠
一定进入 Context

一个完整 Memory System 至少有三个 Decision Layer：



这也呼应 Day04 的 Context Builder：

Context 是 Runtime 在某个时刻生成的 Snapshot，而不是完整 State。

Memory 也一样：

State 很大，但 Context 只投影当前需要的部分。

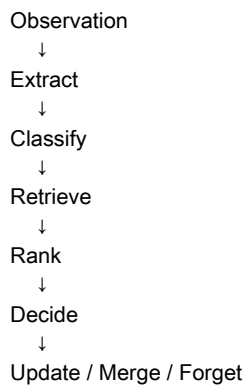
Memory 是否可以看成 Agent

用户在学习中提出：

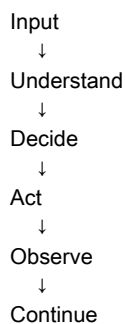
整体这节学下来，感觉 Memory 也完全可以是一个 Agent。

这个感觉是对的，但要区分职责目标。

Memory System 的行为链路已经很像一个小 Agent：



Agent 的链路则是：



从系统行为看，Memory System 确实具备“感知 → 判断 → 行动”的结构。

但它和 Agent 的目标不同：

Memory
→ 管理长期 State

Agent
→ 为了完成 Goal 做决策和行动

所以更准确地说：

Memory System 是 Agent Runtime 中一个具有自治决策能力的子系统。

它的 Action 主要是 State Mutation，而不是外部世界的 Tool Action。

这会连接到后续学习 Planner、Memory Manager、Reflection、Critic、Researcher、Browser Agent 时的更深问题：

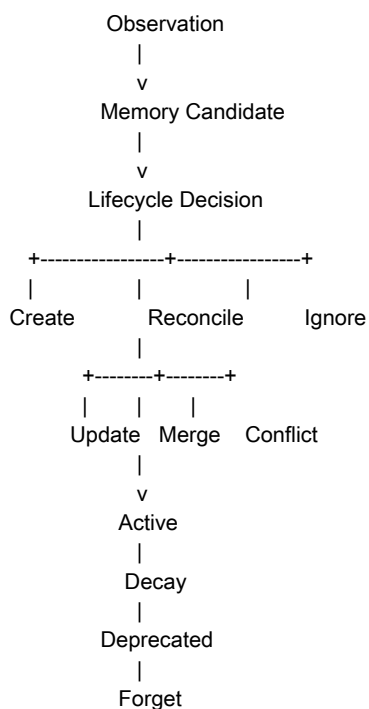
到底什么东西才值得叫 Agent？是拥有 LLM 就叫 Agent，还是拥有独立 Decision Loop 就可以叫 Agent？

Part C 最终模型

Part C 最终不要记成：

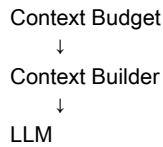
Create
Update
Merge
Decay
Forget

而应该记成：



再接到 Part B：

Memory Lifecycle
↓
Memory Store
↓
Retriever
↓
Ranker
↓



Day06 到这里形成完整闭环：

Lifecycle 决定 Memory “还能不能活”，Retrieval 决定“当前要不要想起”，Context Builder 决定“怎么告诉 LLM”。

本 Part 核心知识点

- Memory Lifecycle 不是 CRUD，而是 Long-term State Reconciliation。
- Memory 是 State，不是 Conversation Event。
- Memory Create 是 State Promotion，不是简单 save。
- Extraction 不等于 Create；Candidate 还需要经过 Runtime Decision。
- Create Decision 至少要考虑长期性、稳定性、可复用性、Scope、敏感性、已有 Memory。
- Importance 表示长期价值，Confidence 表示可信度，Recency 表示时间新鲜度或使用新鲜度。
- Semantic Similarity 只能判断相关，不能判断 State Identity。
- 判断是否 Update 需要 Type、Slot、Entity、Scope、Temporal Signal 和 LLM Judgment。
- Update 是同一个 State Slot 的值发生变化；Merge 是多条信息合成更完整状态。
- Conflict 不一定是错误，而是当前信息不足以确定 State Relationship。
- Decay 是有效性下降，不是删除。
- Forget 是逻辑生命周期变化，不等于物理删除。
- Memory Store 可以包含 Current State 和 History / Audit Log。
- LLM 负责语义判断，Policy 负责硬约束，Runtime 负责最终状态变化。
- Lifecycle Validity、Retrieval Relevance、Context Projection 是三个不同问题。
- Active Memory 不代表一定进入当前 Context。

写书 TODO

Part C 可以整理成书中的 Memory Lifecycle 章节，建议保留以下几个层次。

1. Memory Create

解释为什么：

Conversation
≠
Memory

以及：

Observation
→ Memory Candidate
→ Create Decision
→ Long-term State

重点强调：

Memory Create 本质是 State Promotion。

2. Memory Update / Merge

重点解释：

Similarity
≠
Same Memory

需要结合：

Type
Slot
Entity
Scope
Temporal Signal
LLM Judgment

判断：

Create
Update
Merge
Conflict

3. Memory Decay / Forget

解释：

Decay
≠
Delete

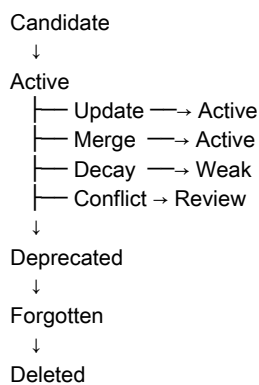
Forget
≠
Physical Delete

并引出：

Confidence
Importance
Recency
Access Pattern
Type
Scope
Explicit Contradiction
Policy

4. Memory State Machine

可以把 Lifecycle 抽象为：



5. Memory Lifecycle Engine

最终将：

LLM Judgment
+
Policy / Rule
+
Runtime

组合成 Lifecycle Decision Engine。

写书素材

素材 1：Memory 不是 CRUD

Memory Lifecycle 不是简单的 Create / Update / Delete，而是 Long-term State Reconciliation。

Observation
+
Existing Memory
↓
State Reconciliation
↓
New Long-term State

素材 2：Similarity 不能决定 Memory Identity

Semantic Similarity 只能回答“它们是否相关”，不能直接回答“它们是不是同一个 State”。

因此需要：

Similarity
+
Type
+
Slot
+
Entity
+
Scope
+
Temporal Signal

素材 3：Memory 有“现在”和“过去”

Current Memory State
+
Memory History

分别回答：

Current State
→ 现在是什么？

History
→ 为什么现在是这样？

例如：

History:

Vue
↓
User explicitly switched
↓

React

Current State:

frontend_framework = React

因此：

Memory System 不只管理“当前记忆”，还可以管理“记忆如何演变”。

本 Part 核心认知升级

Part C 最大的认知升级，是从：

Memory 就是长期保存的信息。

升级到：

Memory 是带有生命周期的 Long-term State。

进一步升级为：

Memory
=
State
+
Identity
+
Scope
+
Lifecycle
+
History

再进一步：

Memory System
=
Lifecycle
+
Retrieval
+
Projection

其中：

Lifecycle
→ 这条 Memory 现在还成立吗？

Retrieval
→ 当前任务需要它吗？

Projection
→ 应该以什么形式告诉 LLM？

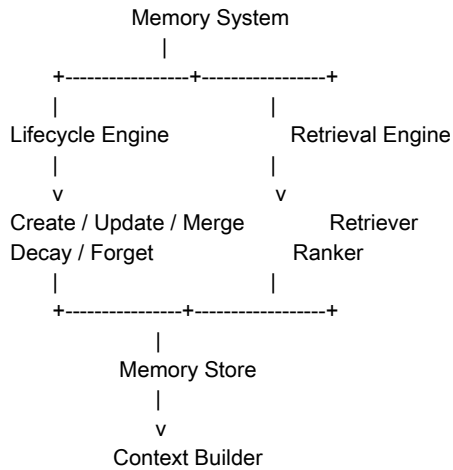
这三个问题不能混在一起。

工业级实现

工业级 Memory 不建议直接设计成：

```
memory.save()  
memory.search()
```


更合理的是分层：



同时：

LLM
→ Semantic Judgment

Policy / Rule
→ Hard Constraints

Runtime
→ State Transition

Store
→ Persistence

不要让：

LLM → 直接修改 Memory DB

而应该：

LLM
↓
Decision / Intent
↓
Runtime Validation
↓
State Mutation

Mini Runtime 第一版不需要一次性实现所有工业能力。可以先做：

Candidate Extraction
↓
Rule-based Lifecycle
↓
Memory Store
↓
Simple Retrieval
↓
Context Builder

然后逐步升级：

v1 Rule
v2 LLM Extraction
v3 Semantic Retrieval
v4 Hybrid Retrieval
v5 Lifecycle Decision Engine
v6 Conflict Resolution
v7 Decay / Forget Policy

架构上预留决策层，不代表第一版必须把所有工业能力实现出来。

知识地图

目前 Day06 已经形成：

```
Day06 Memory
|
+-- Part A: Memory Foundation
|   +-- Conversation
|   +-- Observation
|   +-- Memory
|   +-- Long-term State
|   +-- Memory Extraction
|
+-- Part B: Memory Architecture
|   +-- Memory Entity
|   +-- Memory Store
|   +-- Embedding
|   +-- Vector Search
|   +-- Vector DB
|   +-- Retriever
|   +-- Ranker
|   +-- Hybrid Retrieval
|   +-- Memory Projection
|   +-- Memory vs RAG
|   +-- Enterprise Knowledge Base
|
+-- Part C: Memory Lifecycle
|   +-- Create
|   +-- Update
|   +-- Merge
|   +-- Conflict
|   +-- Decay
|   +-- Forget
|   +-- Lifecycle State Machine
|   +-- Rule vs LLM
|   +-- Current State + History
|
+-- Part D: Memory x Context Builder
|
+-- Part E: Mini Memory Runtime
|
+-- Part F: Industrial Memory Mapping
```

面试视角

如果面试官问：

Memory Lifecycle 和 CRUD 有什么区别？

建议回答：

Memory Lifecycle 不是简单的数据 CRUD，而是根据新的 Observation 对长期 State 进行 Reconciliation。它需要判断 Create、Update、Merge、Conflict、Decay 和 Forget。

为什么不能只靠 Vector Similarity 做 Memory Update？

建议回答：

Similarity 只能判断语义相关性，不能判断是否属于同一个 State。还需要结合 Type、Slot、Entity、Scope、Temporal Signal 等结构化信息。

Update 和 Merge 有什么区别？

建议回答：

Update 是同一个 State Slot 的当前值发生变化，例如 frontend_framework 从 Vue 变为 React。Merge 是多条相关 Memory 合并成一个更完整的长期状态，例如 “works remotely” 和 “works remotely from Japan” 合并成更准确的状态。

Forget 和 Delete 有什么区别？

建议回答：

Forget 更偏向逻辑生命周期变化，即 Memory 不再作为当前有效状态参与正常 Retrieval；Delete 是物理数据删除。工业系统通常会保留历史或审计信息。

LLM 在 Memory Lifecycle 中负责什么？

建议回答：

LLM 更适合语义提取、冲突识别和状态理解；Policy 负责硬约束，Runtime 负责最终状态变更和持久化。LLM Output 不等于 State Mutation。

Active Memory 是否一定进入 Context？

建议回答：

不一定。Lifecycle Validity 只回答 Memory 当前是否仍然有效；Retrieval Relevance 才回答它对当前任务是否相关；Projection 决定它应该以什么形式进入 LLM Context。

Memory 是不是也可以看成一个 Agent？

建议回答：

Memory System 在行为上确实具备 Extract、Retrieve、Decision、State Mutation 等类似 Agent 的闭环，但它的目标不是完成开放式 Goal，而是维护 Long-term State。因此更准确地说，它是 Agent Runtime 中一个具有自治决策能力的子系统，而不一定要单独称为 Agent。

本章思考题

为什么 Memory Create 本质上是一次 State Promotion？

为什么 Memory Extraction 不等于 Memory Create？

为什么 Semantic Similarity 高，不一定应该 Update？

Type + Slot + Scope 为什么能够帮助判断两个 Memory 是否属于同一个 State？

为什么 Scope 能避免 Memory 污染？

Update 和 Merge 的根本区别是什么？

为什么 Conflict 不一定代表错误？

为什么 Historical Memory 和 Current State 不应该完全用同一种存储语义？

为什么 Confidence、Importance、Recency 不能合并成一个简单的 score？

为什么 Active Memory 不代表它一定应该进入当前 Context？

为什么 Forget 和 Physical Delete 应该解耦？

为什么 Memory Lifecycle 更像 State Reconciliation，而不是 CRUD？

为什么 LLM 不应该直接修改 Memory Store ?

如果一个 Memory 长期没有被访问，但它本身是稳定的 User Profile，它是否应该被 Forget ? 为什么 ?

简单 Agent 应该直接上 Event Sourcing 吗 ? 为什么 ?

前置问题回收

Part C 已经解决：

- Memory 什么时候 Create
- Memory Extraction 和 Create 的区别
- Create 为什么是 State Promotion
- Memory Update / Merge 怎么判断
- Vue 到 React 这种偏好变化怎么处理
- Similarity 为什么不能决定 State Identity
- Type、Slot、Entity、Scope、Temporal Signal 的作用
- Conflict 为什么不一定是错误
- Memory 为什么需要 Decay
- Confidence、Importance、Recency 的语义区别
- Memory 什么时候 Forget
- Forget 和 Physical Delete 的区别
- Lifecycle 是 Rule 还是 LLM 驱动
- LLM、Policy、Runtime 在 Lifecycle Engine 中如何分工
- Current State + History / Audit Log 为什么有价值
- Memory Lifecycle、Retrieval、Projection 的职责边界
- Memory System 为什么像 Agent Runtime 中的自治子系统

仍然留到后续：

- Confidence 如何具体计算
- Importance 如何具体计算
- Recency / Decay 是否需要数学模型
- Memory Conflict Resolution 如何实现
- Current State + History 的具体数据结构是什么
- 并发情况下两个 Runtime 同时 Update Memory 怎么处理
- Memory Lifecycle 和 Memory Retrieval 如何在代码中协同
- 一个真实 Mini Memory Runtime 应该如何落成代码

这些问题更适合在 Part E Mini Memory Runtime 中给出工程答案。

下一节学习计划

下一节进入：

Day06 Part D：Memory x Context Builder

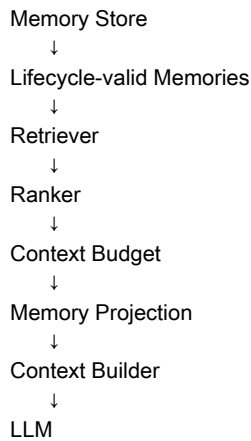
Part C 解决的是：

Memory 怎么产生、变化、失效。

Part D 转而解决：

已经存在的 Memory，Runtime 到底如何把它变成当前 LLM 能消费的 Context？

重点链路：



Part D 会重新连接 Day04 的：

Context Projection
Compression
Token Budget
Priority
Assembly
Snapshot

最终要回答：

为什么“Memory 已经找到”仍然不代表它应该直接进入 Prompt？

Part C 最终一句话

Memory 不是“被保存的信息”，而是 Runtime 持续维护的一组、带有 Identity、Scope、Lifecycle 和 History 的 Long-term State；Create / Update / Merge / Decay / Forget 本质上是在持续完成 State Reconciliation。